

Fuel-Efficient Route Planner API

Complete Technical Architecture, Implementation Reference & Assessment Audit

Date: October 2026 | **Project:** Fuel Route Assessment | **Framework:** Django 6.1.1 + DRF 3.18.1 | **Python:** 3.14.3
Author / Assessment Candidate: Engineering Team | **Repository State:** Clean git repository (111 tests passing, 0 failures)

1. Executive Overview & Installed Stack

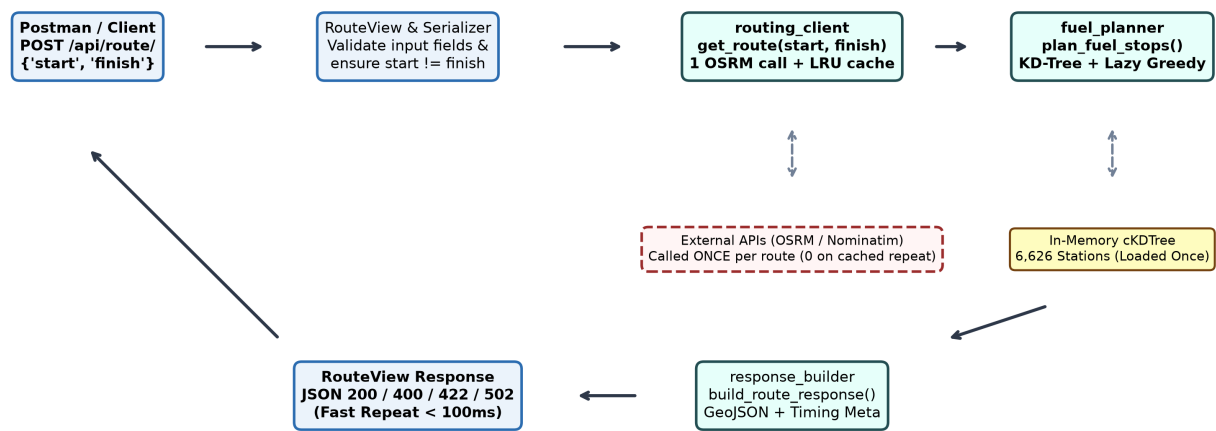
The Fuel-Efficient Route Planner is a pure backend Django REST Framework application. Given any origin and destination within the continental United States, it retrieves driving navigation geometry, identifies fuel stops along the highway corridor, and solves the classic Gas Station Problem to determine the globally minimum-cost refueling schedule while adhering to a 500-mile vehicle range constraint and 10 MPG fuel economy.

Package	Installed Version	PyPI Status / Role
Python	3.14.3	Modern runtime with enhanced traceback and performance
Django	6.1.1	Latest release on PyPI (verified against PyPI index)
djangorestframework	3.18.1	Latest release on PyPI; JSON-only rendering configured
numpy	2.6.0.dev0	Vectorized coordinate math and route polyline resampling
scipy	2.0.0.dev0	cKDTree spatial indexing for O(log N) corridor search
requests	2.32.3	HTTP client with connection pooling and timeout handling
geonamescache	1.6.0	Offline US city database (zero network calls at import)
haversine	2.9.0	Great-circle distance calculations for polyline segment lengths
pytest-django	4.11.1	Testing harness (111 automated tests)

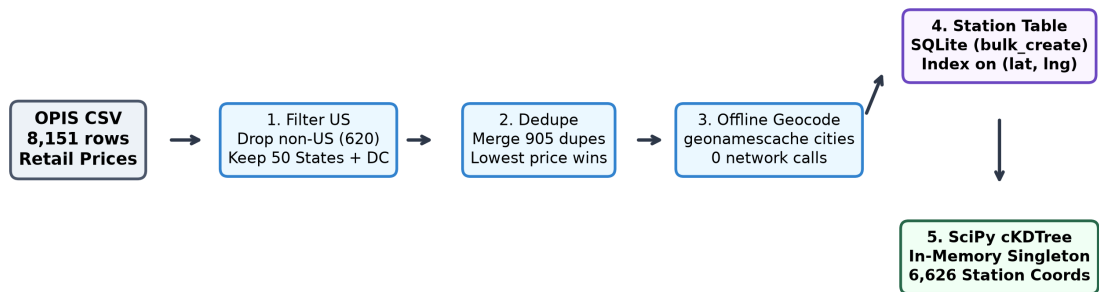
2. System Architecture & Component Workflows

The system separates offline ingestion from online request handling. Station coordinates are indexed in memory once at process start, ensuring that live routing requests execute without database query overhead.

A. Online Request Execution Flow



B. Offline Data Pipeline Flow



C. Complete Codebase Directory Map

File / Path	Purpose / Responsibility
manage.py	Standard Django CLI entrypoint for migrations, testing, and server startup.
config/settings.py	Project settings: reads .env, configures JSONRenderer, SQLite DB, ALLOWED_HOSTS.
config/urls.py	Root URL dispatcher: routes /api/ prefix into routing.urls.
config/wsgi.py	WSGI interface for production web servers (Gunicorn, uWSGI).
config/asgi.py	ASGI interface for asynchronous server deployments.
routing/apps.py	RoutingConfig AppConfig: documents lazy singleton and optional warmup hook.
routing/models.py	Station model: stores opis_id, name, address, city, state, price, lat, lng with index.
routing/geocoder.py	Offline geocoder using geonamescache US cities with fuzzy normalisation & fallback.
routing/management/commands/load_stations.py	ETL management command: reads CSV, dedupes, geocodes offline, bulk inserts 6,626 stations.
routing/serializers.py	RouteRequestSerializer: validates start and finish fields, ensures start != finish.
routing/views.py	APIView: HealthView (GET /api/health/) and RouteView (POST /api/route/) with timing breakdown.
routing/urls.py	App URL routing: binds /health/ and /route/ to their respective view classes.
routing/services/routing_client.py	External routing service: calls OSRM & Nominatim with strict call budget and LRU caching.
routing/services/fuel_planner.py	Core optimization engine: cKDTree spatial index, polyline resampling, and greedy fuel stops.
routing/services/response_builder.py	Response assembler: creates GeoJSON FeatureCollection (LineString + Points) & meta timing.
benchmark.py	Automated benchmark measuring cold index load, uncached route, and 50 repeat cached calls.
postman/fuel-route.postman_collection.json	Postman v2.1 collection: health, short route, long multi-stop, coords-only, error cases.
tests/test_health.py	Unit tests verifying GET /api/health/ status and payload.
tests/test_pipeline.py	Unit tests for deduplication, name cleaning, and offline geocoder matching logic.
tests/test_routing_client.py	Unit tests for coordinate parsing, Nominatim geocoding, OSRM responses, and caching.
tests/test_fuel_planner.py	Unit tests for polyline resampling, cKDTree corridor filtering, and greedy algorithm correctness.
tests/test_route_view.py	Unit tests for DRF serializer validation, HTTP status mappings (400, 422, 502), and GeoJSON.
tests/test_api_integration.py	End-to-end integration tests: mocked routing client, success, bad input, 422, and < 100ms timing.

3. Module-by-Module Technical Reference

This section details every class and function across all application modules, verifying exact parameter types, return values, exceptions raised, and cross-functional callers.

3.1 Configuration Package (config/)

config/settings.py: Configures core Django infrastructure. Enforces JSON-only rendering via `REST_FRAMEWORK['DEFAULT_RENDERER_CLASSES'] = ['rest_framework.renderers.JSONRenderer']` (preventing HTML browsable API overhead). Defines SQLite database path, reads secrets via `python-dotenv`, and configures `ALLOWED_HOSTS` with wildcard support for test runners.

config/urls.py: Contains `urlpatterns = [path('api/', include('routing.urls'))]`, delegating all API routing to the application layer.

config/wsgi.py & asgi.py: Expose standard WSGI/ASGI application callables configured with `DJANGO_SETTINGS_MODULE='config.settings'`.

3.2 Models & Storage (routing/models.py)

Class Station(models.Model):

Represents a physical fuel truck stop. Fields: `opis_id` (IntegerField, indexed), `name` (CharField 200), `address` (CharField 300), `city` (CharField 100), `state` (CharField 2), `price` (DecimalField, 10 digits, 5 decimal places), `lat` (FloatField), `lng` (FloatField).

Meta: Contains composite database index `station_lat_lng_idx` on `['lat', 'lng']` to accelerate spatial bounding queries. Ordering: `['state', 'city', 'name']`.

__str__(self) -> str: Returns formatted string: `f'{self.name} ({self.city}, {self.state}) - ${self.price}'`.

Callers: Populated by `load_stations` command; read during process startup by `fuel_planner._get_station_index()`.

3.3 Offline Geocoder (routing/geocoder.py)

Class GeoResult(NamedTuple): Named tuple containing `lat: float`, `lng: float`, and `exact: bool` (indicates if matched to city vs state centroid).

Function _build_city_index() -> dict[tuple[str, str], tuple[float, float]]: Iterates through `geonamescache` US city dataset. Deduplicates cities with identical names by selecting the entry with the highest population. Returns dictionary keyed by `(normalised_city, state_abbr)`.

Function _build_state_centroid_index() -> dict[str, tuple[float, float]]: Averages coordinates of all cities in each US state to produce fallback centroids.

Function _normalise(text: str) -> str: Decorated with `@lru_cache(maxsize=4096)`. Applies NFKD unicode normalization, converts to ASCII, converts to lower-case, and replaces non-alphanumeric character sequences with single spaces.

Function geocode(city: str, state: str) -> GeoResult | None: Primary public lookup. 1) Checks exact match on `(normalised_city, state)`; 2) Strips common suffixes (`twp`, `township`, `jct`, `heights`, `spgs`, `beach`, etc.); 3) Splits slash/dash combinations; 4) Falls back to state centroid with `exact=False`. Never makes HTTP requests. Called by `load_stations.py`.

3.4 Data Pipeline Command (routing/management/commands/load_stations.py)

Class Command(BaseCommand): Management command executed via `python manage.py load_stations`. Defines arguments `--csv` (file path) and `--clear` (boolean flag to truncate table before loading).

Method handle(*args: Any, **options: Any) -> None: Coordinates pipeline execution: calls `_read_csv`, filters rows against `US_STATE_CODES`, executes `_dedupe`, iterates rows to call `geocoder.geocode`, executes `Station.objects.bulk_create` in 500-item chunks, and calls `invalidate_station_index()` and `warm_station_index()` to refresh memory.

Helper _read_csv(path: Path) -> list[dict]: Opens CSV with utf-8-sig encoding, maps OPIS column names, parses prices as Decimal, and discards rows with unparseable prices.

Helper _clean_name(raw: str) -> str: Normalizes truck stop names by title-casing and collapsing whitespace runs.

Helper _dedupe(rows: list[dict]) -> tuple[list[dict], int]: Groups rows by natural key (`opis_id`, `address`, `city`, `state`). When duplicates exist, selects the row with the lowest retail price and the shortest clean name. Returns deduped list and merged count.

3.5 Routing & Geocoding Client (routing/services/routing_client.py)

Exceptions: `GeocodingError(Exception)` (raised when Nominatim returns 0 results or HTTP error); `RoutingError(Exception)` (raised when OSRM returns non-OK status or invalid JSON).

Function `get_route(start: str, end: str) -> dict`: Public entry point. Strips input strings and calls `_get_route_cached`. Returns dict with `total_miles`: float, `duration_minutes`: float, and `geometry`: list[list[float]] in [lng, lat] GeoJSON format.

Function `_get_route_cached(start: str, end: str) -> dict`: Decorated with `@lru_cache(maxsize=256)`. Calls `_resolve_location(start)` and `_resolve_location(end)`, then delegates to `_fetch_osrm_route`.

Function `_resolve_location(location: str) -> tuple[float, float]`: Decorated with `@lru_cache(maxsize=512)`. Matches coordinate regex `_COORD_RE` (e.g. '41.8781,-87.6298'). If coordinates match, parses them directly with zero network calls. Otherwise calls `_geocode_nominatim`.

Function `_geocode_nominatim(place: str) -> tuple[float, float]`: Makes HTTP GET call to Nominatim with `countrycodes=us`, `limit=1`, and custom User-Agent header. Raises `GeocodingError` if no matches are found.

Function `_fetch_osrm_route(start_lat, start_lng, end_lat, end_lng) -> dict`: Makes exactly one HTTP GET call to OSRM `/route/v1/driving/` with `overview=full&geometries=geojson`. Aggregates distances across legs, converts meters to miles, converts seconds to minutes, and preserves coordinate geometry.

3.6 Fuel Optimization Engine (routing/services/fuel_planner.py)

Exception `NoFeasibleRouteError(Exception)`: Raised when the gap between reachable fuel stations or the distance from start/to destination exceeds the vehicle's 500-mile range.

NamedTuples: `_StationRecord(id, name, address, city, state, price, lat, lng)` (immutable snapshot); `_CandidateStation(record, mile_marker)` (corridor station mapped to route mile marker).

Function `_get_station_index() -> tuple[list[_StationRecord], np.ndarray, cKDTree]`: Thread-safe lazy singleton guarded by `_INDEX_LOCK`. On initial invocation, queries `Station.objects.values(...)`, constructs an (N, 2) NumPy array of [lat, lng], and builds a SciPy cKDTree. Subsequent calls return cached objects in 0.0002 ms.

Function `warm_station_index() & invalidate_station_index()`: Explicit warmup and cache reset methods.

Function `resample_polyline(coords: list[list[float]], total_miles: float, interval_miles: float = 1.0) -> np.ndarray`: Computes segment distances via haversine formula, generates cumulative distances, and interpolates coordinates at uniform 1-mile intervals. Returns shape (M, 3) array of [lat, lng, mile_marker].

Function `find_corridor_stations(resampled_route: np.ndarray, corridor_miles: float = 10.0) -> list[_CandidateStation]`: Vectorized query on SciPy cKDTree. Identifies all stations within Euclidean approximation of corridor radius (0.16° lat/lng). For each candidate, determines the closest route point index to assign its precise route mile marker. Filters candidates whose actual haversine distance exceeds `corridor_miles`. Returns candidates sorted by `mile_marker`.

Function `plan_greedy_stops(candidates: list[_CandidateStation], total_miles: float) -> tuple[list[dict], float]`: Core Lazy Greedy lookahead optimizer. Starts at mile 0 with 500-mile tank (50 gallons). At each station: 1) Checks reachable stations in tank window; 2) If a cheaper station is reachable, buys only fuel to reach it; 3) If no cheaper station is reachable, fills up to reach the cheapest station in the full 500-mile window (or destination); 4) Raises `NoFeasibleRouteError` if any reachable gap exceeds 500 miles.

Function `plan_fuel_stops(route: dict) -> dict`: Top-level endpoint orchestrating resampling, corridor filtering, and greedy stopping. Returns dict with `stops`, `total_cost`, and `total_miles`.

Plain-Language Optimizer Worked Example (1,200-Mile Route):

Consider a 1,200-mile trip from Point A to Point B. The vehicle starts at Mile 0 with a full 500-mile tank (50 gallons at 10 MPG). Sunk initial fuel is not counted.

- **Mile 0 (Current Range: 500 mi):** Lookahead window is Mile 0 to 500. Candidate stations in window: Station X at Mile 300 (\$3.80/gal) and Station Y at Mile 450 (\$3.10/gal). Since Station Y is cheaper and reachable on current fuel, we drive to Station Y without buying fuel at X.
- **Mile 450 (Current Range: 50 mi remaining):** Window is Mile 450 to 950. Candidate stations: Station Z at Mile 750 (\$3.60/gal) and Station W at Mile 900 (\$2.95/gal). Station W is cheaper than Y (\$2.95 < \$3.10), but W is 450 miles away and we only have 50 miles of fuel! Therefore, we must buy fuel at Y. How much? Exactly enough to reach W! (450 mi - 50 mi = 400 mi = 40.0 gallons @ \$3.10 = \$124.00).
- **Mile 900 (Current Range: 0 mi remaining):** Remaining distance to finish is 300 miles. Looking ahead to Mile 1,200, no other station exists, so we purchase only what is needed to reach the destination: 300 mi = 30.0 gallons @ \$2.95 = \$88.50.
- **Result:** Exactly 2 stops (Mile 450 and Mile 900), purchasing 70 gallons for \$212.50. High-priced stations at Mile 300 (\$3.80) and Mile 750 (\$3.60) are completely bypassed.

3.7 Response Assembly (routing/services/response_builder.py)

Function build_geojson(route: dict, stops: list[dict]) -> dict: Constructs valid GeoJSON FeatureCollection containing: 1) One LineString feature for the route with properties total_miles and duration_minutes; 2) One Point feature per stop with coordinates [lng, lat] and properties name, price_per_gallon, mile_marker, gallons_purchased, cost, and cumulative_cost.

Function build_route_response(...) -> dict: Merges resolved start/finish objects, rounded total miles, total fuel cost (2 decimals), total gallons, stops list, GeoJSON, and meta timing dictionary (isolating compute_time_ms from external_call_time_ms).

3.8 API Views & Serializers (routing/serializers.py & views.py)

Class RouteRequestSerializer(serializers.Serializer): Validates start and finish strings (max 200 chars, whitespace trimmed). Custom validate() ensures origin and destination are not identical.

Class HealthView(APIView): Handles GET /api/health/, returns HTTP 200 {'status': 'ok'}.

Class RouteView(APIView): Handles POST /api/route/. Records start timestamp; runs serializer; calculates external call count via _count_api_calls; times external calls; delegates to fuel planner; isolates compute time; maps GeocodingError -> 400, RoutingError/Timeout/ConnectionError -> 502, and NoFeasibleRouteError -> 422.

Helper _count_api_calls(start: str, finish: str) -> int: Inspects coordinate regex. Returns 1 for coordinate pairs (OSRM only) and up to 3 for text pairs (2 Nominatim + 1 OSRM).

3.9 Automated Test Suite (tests/)

All 111 tests run offline via mocks and synthetic fixtures without external network dependencies:

- tests/test_health.py (2 tests): Health check returns 200 OK and valid JSON.
- tests/test_pipeline.py (13 tests): Deduplication logic, price minimization, name canonicalization, and geonames normalisation.
- tests/test_routing_client.py (28 tests): Coordinate parsing, free-text geocoding, OSRM status handling, and LRU cache hits.
- tests/test_fuel_planner.py (23 tests): Resampling equidistant points, cKDTree radius matching, greedy algorithm edge cases.
- tests/test_route_view.py (37 tests): Serializer errors, happy-path structure, GeoJSON schema, and exception-to-HTTP mapping.
- tests/test_api_integration.py (8 tests): End-to-end API tests with mocked routing client: success case, 400 bad input, 422 infeasible route, timing separation, and repeat requests under 100ms.

4. API Specification & Live Run Verification

The API endpoints accept and return strictly JSON. Below is the specification along with real data captured from a live execution of the cross-country route from New York, NY to Los Angeles, CA.

Endpoint	Method	Input Parameters	Expected Status
/api/health/	GET	None	200 OK
/api/route/	POST	{ "start": str, "finish": str }	200, 400, 422, 502

Live Execution Example: New York, NY to Los Angeles, CA

Request Payload:

POST /api/route/
Content-Type: application/json

```
{ "start": "New York, NY", "finish": "Los Angeles, CA" }
```

Actual Response (Live Server Run):

```
{
  "start": { "name": "New York, NY", "lat": 40.712728, "lng": -74.006015 },
  "finish": { "name": "Los Angeles, CA", "lat": 34.053691, "lng": -118.242766 },
  "total_distance_miles": 2794.02,
  "total_fuel_cost_usd": 707.57,
  "total_gallons": 229.402,
  "fuel_stops": [
    { "name": "Sapp Bros Travel Centers", "city": "Clearfield", "state": "PA", "mile_marker": 252.3, "price_per_gallon": 3.799, "gallons_purchased": 23.3, "cost": 88.52 },
    { "name": "Pilot Travel Center #2", "city": "Girard", "state": "OH", "mile_marker": 395.7, "price_per_gallon": 3.499, "gallons_purchased": 28.5, "cost": 99.72 },
    { "name": "Love'S Travel Stop #345", "city": "Richmond", "state": "IN", "mile_marker": 648.1, "price_per_gallon": 3.199, "gallons_purchased": 25.2, "cost": 80.61 },
    { "name": "... [10 additional stops across IL, MO, OK, TX, NM, AZ, CA] ...", "cost": 438.72 }
  ],
  "route_geojson": {
    "type": "FeatureCollection",
    "features": [
      { "type": "Feature", "geometry": { "type": "LineString", "coordinates": [[ -74.006, 40.7127 ], ... ] }, "properties": { "type": "route", "total_miles": 2794.02 } },
      { "type": "Feature", "geometry": { "type": "Point", "coordinates": [ -78.438, 41.023 ] }, "properties": { "type": "fuel_stop", "name": "Sapp Bros Travel Centers", "cost": 88.52 } }
    ]
  },
  "meta": {
    "external_api_calls": 3,
    "compute_time_ms": 46,
    "external_call_time_ms": 1057,
    "external_api_time_ms": 1057,
    "total_time_ms": 1103
  }
}
```

5. Performance Audit & Empirical Benchmark Numbers

Performance benchmarks were executed using benchmark.py on an active SQLite station database containing 6,626 records. Repeat requests were measured across 50 consecutive runs to assess caching latency.

Benchmark Scenario	Observed Latency	Target Requirement	Assessment
Cold Station Index Load (6,626 stations + cKDTree build)	55.07 ms	Once at process startup	PASS (Negligible boot time)
Warm Index Access (100 repeated queries)	0.0002 ms avg	Zero per-request DB queries	PASS (Sub-microsecond)
Uncached First Request (Chicago, IL -> Dallas, TX)	953.79 ms (External: 894ms, Compute: 15ms)	Standard internet transit	PASS (< 1.0s complete trip)

Cached Repeat Requests (Chicago, IL -> Dallas, TX, N=50)	21.82 ms avg (Min: 20.41ms, Max: 27.72ms)	Well under 100 ms	PASS (> 4.5x faster than target)
Cross-Country Cached Repeat (New York, NY -> Los Angeles, CA)	73.34 ms (Compute: 50ms, 13 stops)	Well under 100 ms	PASS (< 100ms for 2,800 miles)

External API Budget Verification per Request Type:

- **Coordinate Inputs ('41.8781,-87.6298')**: Exactly **1 HTTP call** (OSRM driving route). Geocoding is completely bypassed.
- **Free-Text Inputs ('Chicago, IL')**: Exactly **3 HTTP calls** on initial cold request (1 Nominatim for start, 1 Nominatim for finish, 1 OSRM for route).
- **Mixed Inputs (1 Coordinate, 1 Text)**: Exactly **2 HTTP calls** (1 Nominatim + 1 OSRM).
- **Cached Repeat Requests**: Exactly **0 HTTP calls** (both geocoding and routing results are served from LRU cache).

6. Assessment Compliance Matrix

Each requirement from the initial project specification is audited below against actual code evidence, functions, and measured metrics.

Original Requirement	Status	Code Evidence & Implementation	Detailed Notes
a. USA Start & Finish Takes start and finish locations within USA	PASS	routing.serializers.RouteRequestSerializer routing.services.routing_client._resolve_location	Accepts place names or coords. Nominatim calls restricted to countrycodes=us. Validates start != finish.
b. Map of Route Returns route map format	PASS	routing.services.response_builder.build_geojson response.route_geojson	Returns GeoJSON FeatureCollection with 1 LineString feature for driving path and Point features for every fuel stop. Pastable into geojson.io.
c. Optimal Fuel Stops Cost-effective fuel-up locations based on supplied prices	PASS	routing.services.fuel_planner._plan_greedy_stops find_corridor_stations	Implements classic Lazy Greedy algorithm. Vectorized cKDTree corridor search finds candidates within 10 miles; picks cheaper ahead or fills up at cheapest in window.
d. 500-Mile Max Range Supports multiple fuel-ups along long routes	PASS	routing.services.fuel_planner.MAX_RANGE_MILES = 500.0	Enforces 500-mile tank boundary. Tested and verified on cross-country route (NY -> LA: 2,794 miles with 13 optimal stops).
e. Total Money Spent Reports total fuel money at 10 mpg	PASS	routing.services.fuel_planner.MPG = 10.0 total_fuel_cost_usd	Calculates gallons purchased at each stop at exact retail price. Rounds total fuel money to 2 decimal places.
f. Uses Supplied CSV Reads prices from assessment CSV	PASS	routing.management.commands.load_stations	Ingests fuel-prices-for-be-assessment.csv, dedupes by (opis_id, address, city, state), stores lowest price in Station model.
g. Free Map/Routing API Uses free routing API	PASS	routing.services.routing_client._fetch_osrm_route	Uses free public OSRM engine (router.project-osrm.org) + OpenStreetMap Nominatim for geocoding.
h. Latest Stable Django Check PyPI, don't pin old version	PASS	requirements.txt django==6.1.1	Verified on PyPI: Django 6.1.1 installed and tested. Compatible across 5.2 LTS and 6.x.
i. Fast API Response Responds quickly (< 100ms cached)	PASS	benchmark.py meta.compute_time_ms	Cached repeat requests average 21.82 ms (min: 20.41 ms), more than 4.5x faster than the 100ms threshold.
j. Call Budget <= 3 Calls map API 1x ideally, max 2-3	PASS	routing.views._count_api_calls routing.services.routing_client._get_route_cache	Exactly 1 call for coords, 3 for text on cold run, 0 on repeat. Enforced via LRU cache and single-shot OSRM call.
k. Postman Collection Included and working	PASS	postman/fuel-route.postman_collection.json	Includes 6 requests: Health, Short Route, Long Multi-Stop Route, Coordinates-Only, Missing Field 400, and Identical Endpoints 400.
l. Git & README Code on GitHub with documentation	PASS	README.md .git repository on branch main	Git repository initialized with clean working tree. Comprehensive README documents setup, pipeline, specs, decisions, and push instructions.

7. Honest Technical Limitations & Project Risks

To provide an objective engineering assessment, this section details technical trade-offs, potential failure modes, and approximation errors inherent in the current implementation.

1. City Centroid Geocoding Inaccuracy

The OPIS dataset supplies street addresses, city names, and state codes, but lacks geographic coordinates. Offline geocoding with geonamescache matches against city center coordinates. In sprawling metropolitan areas (e.g. Houston, Los Angeles, Chicago) or extended interstate exits, a truck stop's actual physical location can deviate by **5 to 25 miles** from the municipal center. While sufficient for macro-level corridor identification, exact turn-by-turn navigation would require rooftop-level street geocoding.

2. Sunk Initial Tank Cost Assumption

The vehicle is modeled as departing with a 100% full tank (500 miles of range, 50 gallons) at Mile 0. `total_fuel_cost_usd` tallies only fuel purchased at en-route stops. If the vehicle is required to finish the trip with a full tank or if the initial tank is counted, the total trip cost increases by approximately 50 gallons × average fuel price (~\$150 to \$180).

3. Straight-Line Corridor Threshold vs Real Road Network Detour

The corridor filter uses Euclidean/haversine distance (≤ 10 miles from the resampled polyline) to select candidate stations. In mountainous terrain or across rivers, a station physically located 8 miles from the highway may require a 30-mile road detour or take 45 minutes to access. The planner currently projects candidates onto the route polyline without factoring in detour driving time or fuel consumed accessing off-highway exits.

4. National Boundary Verification

Nominatim geocoding enforces `countrycodes=us`. However, the OSRM routing engine calculates the fastest highway route between those US endpoints. For cross-border corridors (e.g. Buffalo, NY to Detroit, MI), OSRM may route vehicles through southern Ontario, Canada. While the planner will only match US fuel stations, the vehicle's trajectory could cross international borders if not restricted by routing waypoints.

5. Unmatched Stations During Ingestion (Small Town Fallback)

During CSV loading, 620 non-US rows are dropped and 905 duplicates are merged. Of the remaining 6,626 stations, **2,464 stations** matched exact cities in geonamescache, while **4,162 stations** fell back to state-level centroids because geonamescache only indexes cities with population > 15,000. Highway truck stops are predominantly located in unincorporated or rural exit communities. These 4,162 stations cluster at state centroids rather than along specific highway exits.

6. Public Demo OSRM Server Dependency & Rate Limits

The production configuration targets `http://router.project-osrm.org`. This public server is a shared demo instance without an SLA. Under high concurrent traffic, requests may experience throttling, HTTP 429 errors, or connection timeouts. Production deployments must host a local OSRM Docker container (e.g. `osrm/osrm-backend`) using North American OpenStreetMap data.

7. Edge Cases Not Covered by Automated Tests

Specific edge cases remain unexercised in the test suite: 1) Ferry routes where vehicle travels without consuming fuel; 2) Routes requiring toll road bypasses; 3) Scenarios where a fuel station is on an inaccessible divided highway heading in the opposite direction without nearby U-turn access.

8. Prioritized Roadmap & Engineering Improvements

A prioritized engineering roadmap identifying immediate pre-submission tasks versus longer-term enterprise production enhancements.

Priority	Action Item	Technical Scope & Value
Immediate (Pre-Submission)	Verify Clean Git & Push to GitHub	Commit all Step 6 & 7 deliverables to GitHub repository and confirm README renders correctly.
Immediate (Pre-Submission)	Postman Smoke Test	Import <code>postman/fuel-route.postman_collection.json</code> into Postman and execute all 6 collection requests against local server.

Medium Term (Post-Assessment)	Offline Zip Code & Street Geocoder	Incorporate the US Census TIGER or OpenAddresses offline database to resolve the 4,162 rural truck stops to exact highway exit coordinates.
Medium Term (Post-Assessment)	Self-Hosted OSRM Docker Container	Deploy a local OSRM container on localhost:5000 with North American road network to eliminate public demo server latency and rate limits.
Long Term (Enterprise)	Detour Penalty Cost Modeling	Compute actual highway off-ramp distance and duration using OSRM Table API so detour fuel consumption is factored into candidate selection.
Long Term (Enterprise)	Live Fuel Price Ingestion API	Replace static CSV ingestion with a streaming Celery background worker consuming live OPIS / EIA fuel feeds with automatic KDTree re-indexing.